

# Parte 3. Gestor de Memoria

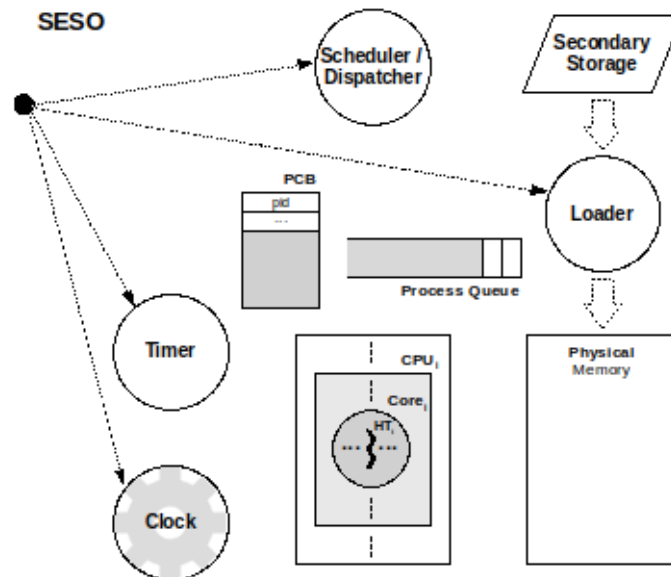
Sistemas Operativos

Curso 2025-2026

Sobre la base del marco de funcionamiento construido en las dos primeras partes en la que habéis desarrollado un **Scheduler** (y **Dispatcher**) con unas determinadas políticas de planificación, queremos construir un sistema de gestión de memoria virtual. Esta parte la realizaremos en dos fases. En la primera fase, construiremos todo el entorno para poder simular el sistema. En la segunda fase, construiremos el propio sistema de gestión de memoria virtual.

## Fase 1: Ampliación de la arquitectura del sistema

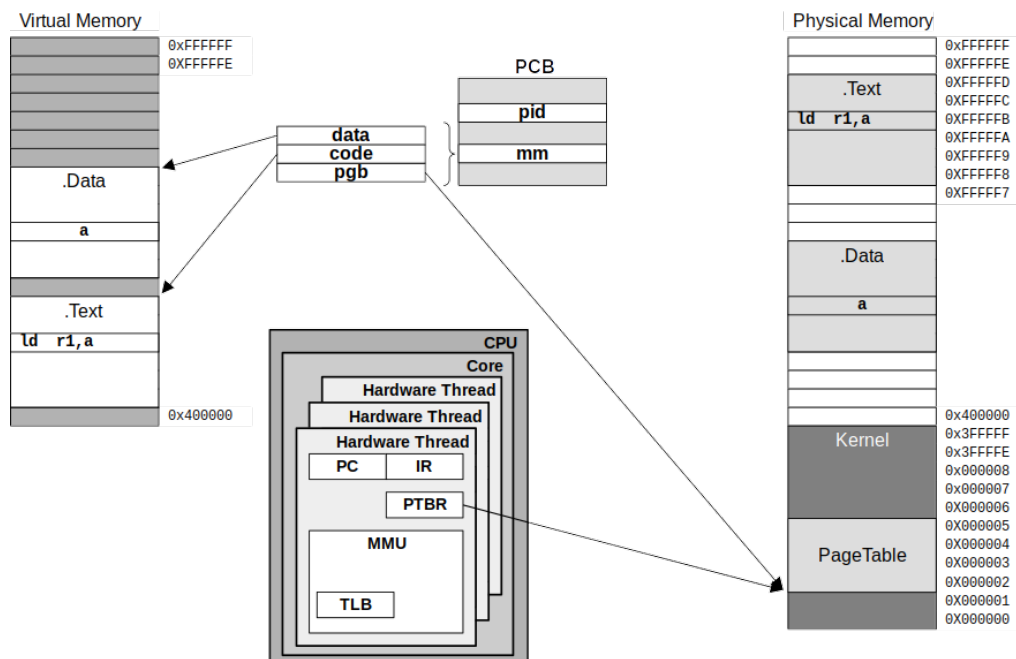
Para ello vamos a modificar parte de lo elaborado y vamos a crear nuevas estructuras de acuerdo al siguiente esquema:



Por un lado vamos a transformar el **Process Generator** en un cargador, **Loader**. La máquina de cómputo, **CPUs**, compuesto por CPUs, cores e hilos hardware, va a ampliar su arquitectura. Por otro lado también crearemos una memoria física, **Physical Memory**.

## El sistema de Memoria Virtual

El sistema de Memoria Virtual va a manejar instrucciones y datos. Para ello vamos a simular unos pequeños programas con unas instrucciones muy básicas, que definiremos en la segunda fase. Este programa se compondrá de sólo dos segmentos: instrucciones (**.text**) y datos (**.data**). Cada hilo hardware tendrá asociado, entre otras cosas, una **MMU**, una **TLB**, un registro apuntador a la tabla de páginas (**PTBR**), un registro de instrucción (**RI**) o un **PC**. La memoria física, **Physical Memory**, tendrá un bus de direcciones de 24 bits y sus posiciones de tamaño de una palabra (4 Bytes). Debes tener en cuenta que el Kernel tiene que tener reservado un tamaño fijo (común a cualquier proceso) que es donde se encontrarán físicamente las tablas de página.



## Funcionamiento

El funcionamiento general debería ser el siguiente:

- En primer lugar se debe crear la memoria física y completar los cores.
- El **Loader**, en vez de generar procesos al azar, leerá programas de un fichero de texto. Utilizaremos ficheros para simular el disco (**Secondary Storage**). Deberá cargarlos en memoria y actualizar las estructuras de datos necesarias (p.e. la tabla de páginas). Ten en cuenta que tanto los segmentos **.text** y **.data**, como las tablas de páginas deben residir en memoria física.

Para ello deberá proceder de la manera siguiente:

- Crear e inicializar el **PCB**.
- Crear la tabla de páginas del proceso en el espacio del Kernel y con un apuntador físico que se guardará en el campo **pgb** del **PCB**.